

IA embarquée pour la perception des véhicules autonomes en logistique hospitalière

PRI 2026-2027 · Projet 2 · Encadrement : Moïse DJOKO-KOUAM
Note de synthèse — version 2, septembre 2026

État des lieux de l'existant, comparaison des modèles et choix technologiques.

1. Le problème, reformulé

Problématique du sujet : concevoir une IA de perception *performante, frugale et apte au déploiement embarqué*, qui transforme les informations des capteurs du véhicule autonome (VA) en une représentation structurée de son environnement.

Le point structurant : le module de perception n'est pas une fin en soi. Ses résultats sont destinés à une **IA de décision**, hors du périmètre du projet. Ce que nous devons livrer, c'est donc un flux d'informations claires, datées et **accompagnées d'un niveau de confiance**, que cette IA de décision pourra exploiter directement.

Information attendue	Produite par
a — Obstacle détecté, position et distance	Caméra (détection) + LiDAR (distance)
b — Personne présente	Caméra (détection)
c — Couloir dégagé ou obstrué	LiDAR + suivi des objets
d — Ascenseur disponible	À préciser (voir §4 et §8)
e — Trajectoire libre ou bloquée	Suivi des objets et de leur vitesse
f — Niveau de confiance	Recalage des scores du modèle (§6)

Le niveau de confiance (f) demande une attention particulière : les modèles de détection ont tendance à être **trop sûrs d'eux**. Un score affiché de 90 % correspond souvent, en réalité, à environ 70 % de bonnes réponses. Il faut donc le corriger avant de le transmettre.

2. Le cadre du projet

2.1 Les décisions prises

Point	Décision
Terrain d'essai	Phase 1 sur une plateforme de test à l'école , avec des couloirs reconstitués
Capteur de distance	LiDAR 2D (un laser qui balaie un plan horizontal et mesure les distances)
Vitesse du robot	Réglable à volonté — le robot est déjà pilotable
Matériel de calcul	Raspberry Pi 5 seul , sans accélérateur : pas de budget supplémentaire
Modèle de détection	YOLOv8n , exécuté sur le processeur du Raspberry Pi (§5)

2.2 La contrainte principale : le Raspberry Pi 5

Le Raspberry Pi 5 est un ordinateur de la taille d'une carte de crédit. Il n'a **ni carte graphique ni puce dédiée à l'IA** : tout le calcul repose sur son processeur à 4 cœurs. C'est la contrainte qui oriente tous les choix de ce document.

Deux points favorables tout de même :

- son processeur accélère efficacement les **modèles compressés** (voir §5.4), ce que ne faisaient pas les générations précédentes de Raspberry Pi ;
- il chauffe sous charge et ralentit alors de lui-même : **un ventilateur est indispensable**, sans quoi les mesures de vitesse perdent toute valeur.

2.3 Le budget de temps

Pour s'arrêter en douceur devant un piéton qui arrive en face, un robot roulant à 1 m/s doit le repérer à **environ 4 mètres**. En visant **10 analyses d'image par seconde** (soit une analyse tous les 10 cm parcourus), et une fois retiré le temps de capture et de traitement autour du modèle, il reste :

Environ 50 ms par image pour le modèle de détection

2.4 La vitesse du robot, notre variable d'ajustement

Puisque la vitesse du robot est réglable, ce budget n'est pas un mur. Si le modèle est plus lent que prévu, **on adapte la vitesse** pour conserver la même marge de sécurité — une analyse tous les 10 cm environ :

Cadence obtenue sur le Raspberry Pi	Vitesse maximale conseillée
12 images/s	1,2 m/s
10 images/s	1,0 m/s
8 images/s	0,8 m/s
5 images/s	0,5 m/s

C'est un vrai atout pour la plateforme de test : le projet reste démontrable quelle que soit la vitesse finalement atteinte par le modèle.

3. État des lieux de l'existant

La logistique hospitalière autonome n'est pas un domaine émergent : plusieurs robots sont déployés depuis plus de dix ans.

Système	Capteurs	Calcul	Approche
Aethon TUG (depuis ~2004)	LiDAR, ultrasons, infrarouge, puis caméra de profondeur	Ordinateur embarqué	Carte du bâtiment + capteurs de distance ; la caméra est arrivée en dernier , en complément
Panasonic HOSPI (depuis ~2013)	Capteurs de distance multiples	Ordinateur embarqué	Carte pré-établie ; certifié selon la norme de sécurité des robots de service
Relay Robotics	LiDAR, caméra de profondeur, ultrasons	Ordinateur embarqué	Circulation en espaces publics fréquentés
Diligent Moxi	LiDAR, caméras	Ordinateur embarqué puissant	Navigation + saisie d'objets + interaction avec le personnel
ORB — Carnegie Mellon, 2025	Multiples	Carte graphique	Trois grands modèles d'IA combinés ; très performant mais très gourmand

Système	Capteurs	Calcul	Approche
Prototypes sur Raspberry Pi (universitaires)	Caméra, souvent seule	Raspberry Pi	Un modèle YOLO seul — 6 à 8 images/s

Trois approches se dégagent : l'industrie résout le problème **avec du matériel puissant** ; la recherche **avec des cartes graphiques** ; les prototypes à bas coût s'arrêtent à la détection brute, sans organiser l'information ni indiquer leur niveau de confiance.

Notre positionnement : produire, sur un matériel à bas coût, une information **structurée et fiable** — ce que les prototypes existants ne font pas encore.

4. Ce que l'on retient de l'existant

	Enseignement	Conséquence pour notre projet
1	Les capteurs de distance d'abord, la caméra ensuite. Aucun robot en production ne se repère avec la caméra seule : tous s'appuient sur des capteurs de distance, et la caméra vient compléter	Le LiDAR 2D fournit les distances et l'espace libre, sans calcul coûteux. Inutile d'ajouter un second modèle d'IA pour estimer les distances à partir de l'image
2	La sécurité des personnes repose sur un capteur certifié, pas sur l'IA. Les normes de sécurité des robots mobiles imposent un scanner laser de sécurité certifié pour détecter les personnes et déclencher l'arrêt	Notre module est un démonstrateur de perception , pas un organe de sécurité. Il n'a pas à être parfait, mais il doit être régulier et honnête sur sa confiance
3	Ce que le bâtiment sait, on le lui demande. Les robots du marché interrogent directement l'ascenseur par le réseau pour savoir s'il est disponible	L'information « ascenseur disponible » relève plutôt d'une liaison avec le bâtiment que de la caméra. Sans objet sur la plateforme de test (§8)
4	Sur Raspberry Pi, un modèle YOLO non optimisé tourne à 6-8 images/s , en dessous de notre cible	L'effort doit porter sur la manière d'exécuter le modèle plus que sur le choix du modèle lui-même (§5.4)

5. Comparaison des modèles de détection

5.1 Les candidats

Modèle	Sortie	Précision*	Vitesse sur Pi 5	Retours d'expérience sur Raspberry Pi	Verdict
SSD-MobileNet / EfficientDet-Lite	2018-2020	22 à 32	—	Nombreux	Écarté : précision trop faible
NanoDet-Plus	2021	~27	—	Limités ; projet peu maintenu	Écarté
YOLOv8n	janv. 2023	37,3	~83 ms (≈ 12 img/s)	Très nombreux : documentation officielle, études universitaires, tutoriels	Retenu
YOLO11n	sept. 2024	39,5	~80 ms	Nombreux	Alternative proche
YOLO26n	janv. 2026	40,9	~67 ms	Encore rares : chiffres surtout issus de l'éditeur	Piste d'évolution
RF-DETR / D-FINE	2024-2025	Les plus élevées	Non documentée	Quasi inexistants	Écartés pour le robot ; utiles hors ligne pour pré-annoter nos images

* Note de précision sur le jeu de données de référence COCO (0 à 100). Vitesses mesurées à 640 pixels avec le moteur d'exécution NCNN. Ces chiffres, issus de la littérature, servent à comparer les modèles entre eux ; les valeurs réelles seront mesurées sur notre Raspberry Pi (§7).

5.2 Pourquoi YOLOv8n

L'argument principal : la maturité et la documentation.

YOLOv8 est utilisé depuis début 2023, soit plus de trois ans et demi. Sur Raspberry Pi 5 en particulier, il a fait l'objet de **nombreuses évaluations indépendantes** — articles universitaires, bancs d'essai publiés, guides de déploiement pas à pas. YOLO26, sorti en janvier 2026, n'a que quelques mois : les chiffres disponibles sur Raspberry Pi proviennent encore essentiellement de l'éditeur lui-même.

Pour un projet de durée limitée, cette différence pèse lourd :

- **Moins de risque.** Chaque difficulté que nous rencontrerons (installation, conversion du modèle, réglages) a très probablement déjà été rencontrée et résolue par d'autres. Avec un modèle récent, nous serions parmi les premiers à les affronter.
- **Des résultats comparables.** Les performances de YOLOv8 sur Raspberry Pi sont déjà publiées : nous pourrions situer nos propres mesures par rapport à des références connues.

L'écart de performance est faible, et en partie rattrapable. YOLO26n est un peu plus précis (+3,6 points) et un peu plus rapide, mais :

- la vitesse reste du même ordre (~83 ms contre ~67 ms) ;
- cette précision est mesurée sur des objets du quotidien (COCO). Nous allons de toute façon **ré-entraîner le modèle sur nos propres images** et nos propres classes d'objets : c'est sur nos couloirs que l'écart comptera, et il reste à mesurer.

Aucune porte n'est fermée. YOLOv8, YOLO11 et YOLO26 s'utilisent avec la même bibliothèque (Ultralytics), avec les mêmes commandes d'entraînement et de conversion. Passer plus tard à un modèle plus récent ne demandera que de **changer le nom du modèle dans le code** : tout le reste du travail (données, entraînement, intégration) sera réutilisable.

5.3 Ce que l'on accepte en choisissant YOLOv8n

Un détecteur propose plusieurs boîtes pour un même objet ; une étape de tri garde ensuite la meilleure. Sur YOLOv8, ce tri prend un peu plus de temps quand la scène contient beaucoup d'objets — le temps de calcul varie donc légèrement selon l'encombrement. YOLO26 a supprimé cette étape.

C'est une **limite connue et maîtrisable** : on plafonne le nombre d'objets traités par image, et on vérifie par la mesure que le temps de calcul reste acceptable **dans les scènes les plus chargées**, pas seulement en moyenne.

5.4 Le levier principal : la manière d'exécuter le modèle

Le même modèle peut tourner avec différents **moteurs d'exécution** (les logiciels qui font tourner le modèle). C'est le choix qui a le plus d'impact :

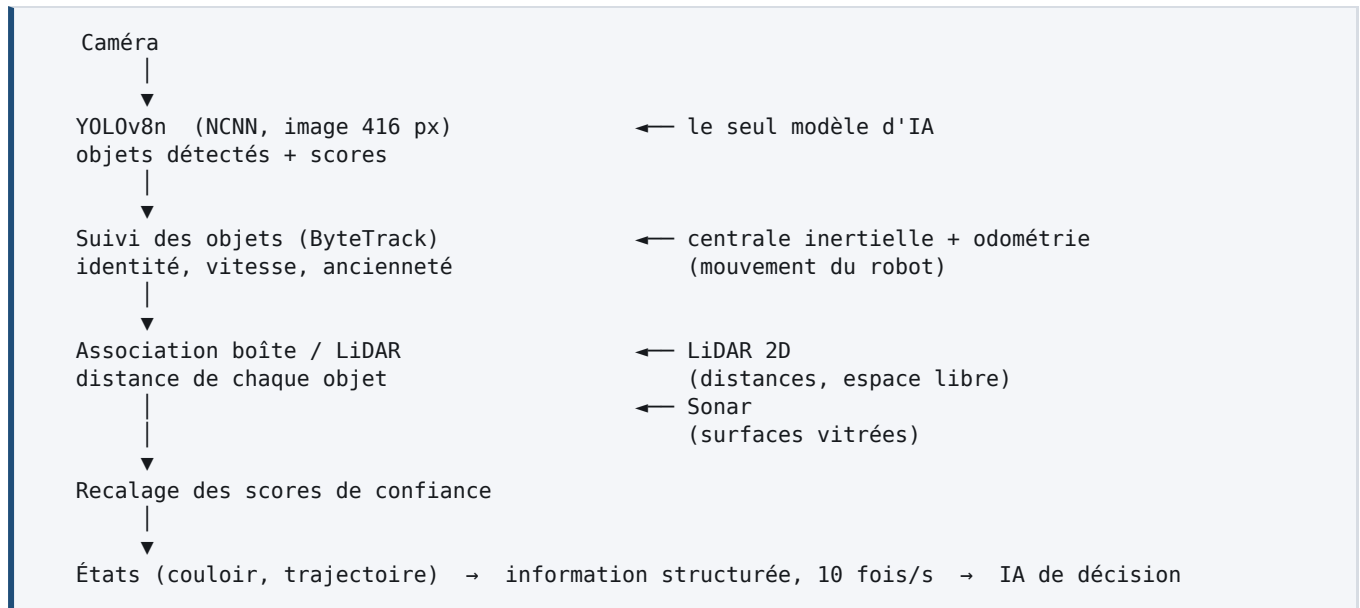
Optimisation	Gain	Principe
Moteur NCNN au lieu de PyTorch	× 3 à × 5	Moteur conçu pour les processeurs de téléphones et de Raspberry Pi. PyTorch sert à entraîner, pas à exécuter rapidement
Image réduite de 640 à 416 pixels	× 2 à × 2,4	Moins de pixels à analyser. Dans un couloir, les obstacles sont proches, donc grands dans l'image : la perte devrait être faible (à vérifier, §7)
Modèle compressé (nombres sur 8 bits au lieu de 32)	× 2	Calculs plus légers, bien pris en charge par le processeur du Pi 5

Objectif : passer des ~83 ms actuels à **25-40 ms par image**, donc dans le budget de 50 ms. Ces gains ne s'additionnent jamais parfaitement : ils seront mesurés un par un.

6. Le choix retenu

Principe : un seul modèle d'IA dans la boucle

Le modèle de détection est le seul élément coûteux en calcul. Toutes les autres fonctions — distance, suivi, état du couloir, confiance — sont assurées par des calculs simples, en tirant parti des capteurs déjà présents sur le robot. C'est ce qui permet de tenir à la fois les exigences de performance, de frugalité et d'embarqué du sujet.



Fonction	Solution	Pourquoi
Détection	YOLOv8n sur le processeur du Pi 5	Maturité, documentation, retours d'expérience (§5.2)
Distance	Association caméra / LiDAR 2D : pour chaque objet détecté, on lit la distance mesurée par le LiDAR dans la même direction	Distance en mètres, précise, sans calcul coûteux
Espace libre	LiDAR 2D	Même un objet que le modèle ne connaît pas est vu comme « pas du sol » : un filet de sécurité naturel
Surfaces vitrées	Sonar	Le verre est invisible pour le LiDAR comme pour la caméra ; le son, lui, rebondit dessus
Suivi	ByteTrack, un algorithme de suivi léger, aidé par la centrale inertielle du robot	Donne une identité et une vitesse à chaque objet. Il conserve les détections incertaines (personne à moitié cachée), ce qui aide à ne rater personne
Confiance	Recalage des scores sur un jeu d'images de contrôle, combiné à la durée de suivi et à la confirmation par le LiDAR	Rend le score honnête, sans alourdir le calcul

7. Plan de validation — phase 1 sur la plateforme de l'école

L'état de l'art oriente les choix ; c'est la mesure qui les confirmera. Quatre étapes :

- ① **Première mesure.** Installer YOLOv8n tel quel sur le Raspberry Pi 5 et mesurer sa vitesse réelle, avec ventilateur et après une période de chauffe. C'est notre point de départ.
- ② **L'expérience prioritaire : taille d'image et détection des personnes.** Sur les couloirs de la plateforme, mesurer la proportion de personnes correctement détectées selon la taille de l'image (640, 416,

320 pixels) et la distance. Si la détection reste bonne à 416 pixels, la faisabilité sur le Raspberry Pi seul est acquise. Sinon, on ajuste la vitesse du robot (§2.4). **Le savoir tôt est la meilleure façon de réduire le risque du projet.**

③ **Nos données.** Photographier les couloirs de la plateforme de jour comme en éclairage réduit, avec des objets représentatifs du milieu hospitalier (chariots, fauteuils, obstacles au sol), avec l'accord des personnes filmées. Un modèle plus gros peut pré-annoter les images sur ordinateur ; il ne reste qu'à les corriger.

④ **Entraînement et optimisation.** Ré-entraîner YOLOv8n sur ces images, appliquer les optimisations du §5.4, puis mesurer à nouveau.

Critères de réussite proposés

Critère	Objectif
Personnes détectées parmi celles présentes	$\geq 95 \%$
Précision globale sur nos classes d'objets	$\geq 60 \%$
Temps de calcul par image (cas courant / scènes chargées)	$\leq 50 \text{ ms} / \leq 100 \text{ ms}$
Taille du modèle	$\leq 15 \text{ Mo}$
Consommation électrique du Raspberry Pi	$\leq 8 \text{ W}$
Écart entre confiance annoncée et confiance réelle	$\leq 5 \%$

8. Points encore ouverts

1. **Ascenseur.** Sans objet sur la plateforme de test. Pour une phase ultérieure en milieu réel : l'information peut-elle être obtenue par une liaison avec l'ascenseur, ou faut-il l'estimer par la caméra ?
2. **GPS et RFID.** Le GPS ne fonctionne pas en intérieur, et le RFID indique un lieu plutôt qu'un obstacle. Sont-ils dans le périmètre de la perception ?
3. **Suite de la plateforme de test.** Un passage en milieu hospitalier réel est-il envisagé après la phase 1 ?

À noter sur la licence. Les modèles Ultralytics, dont YOLOv8, sont diffusés sous licence **AGPL-3.0** : aucune contrainte pour un projet académique publié en accès libre, mais une licence commerciale serait nécessaire en cas d'exploitation industrielle fermée.

Sources principales

- **Systèmes existants** — Aethon TUG / Intel RealSense · Panasonic HOSPI, certification ISO 13482 · Relay Robotics, intégration ascenseur · Diligent Moxi · ORB: Operating Room Bot (arXiv 2509.15600)
- **YOLOv8 sur Raspberry Pi** — Edge AI Engineering : YOLO sur Raspberry Pi · Benchmarking Deep Learning Models for Object Detection on Edge Devices (arXiv 2409.16808) · Quantized YOLOv11 and YOLOv8 on Raspberry Pi 5 (Research Square) · YOLOv8 et RT-DETR : efficacité énergétique en périphérie (Scientific Reports) · YOLO sur Raspberry Pi — guide Ultralytics
- **Modèles comparés** — Ultralytics YOLOv26 · RF-DETR (arXiv 2511.09554) · ByteTrack et autres algorithmes de suivi — comparatif
- **Sécurité** — ISO 3691-4:2020 · Scanners laser de sécurité pour la détection de personnes